FRI

(Fast Reed-Solomon IOP of Proximity)

A learning group for ZK and SNARK application development

Daniel Szego

In: https://www.linkedin.com/in/daniel-szego/

Agenda

- *ZK learning group logistics*
- *Zero knowledge proofs - summary*
- *High level flow of zero knowledge programming*
- *STARK*
- *Reed Solomon Codes*
- *Merkle tree and vector commitment*
- *FRI concepts*
- *FRI workflow*
- *Demo*
- *Challenge*
- *Links, Resources*
- *Q&A*

License: Apache 2.0

Logistics

Every month or two months

Similar structure, similar logistics + ***guest lectures***

Theory:

- Look tables,
- Zk variations: STARKs, FRI
- recursive SNARKs

Programming / platforms:

- Zinc / ZKSync
- Leo

Applications:

- ZK bridges
- ZK Virtual Machines
- ZK Identity / verifiable credential - presentations
- Voting

Discord channel:

https://discord.com/channels/905194001349627914/1329201532628898036

Meetup.com:

https://www.meetup.com/lfdt-hungary/events/305634614/

Repo with all the contents:

https://github.com/LF-Decentralized-Trust-labs/zk-learning-group

 License: Apache 2.0

Zero knowledge proofs - summary

Computation: arithmetic circuit : *C(x, w) -> F*

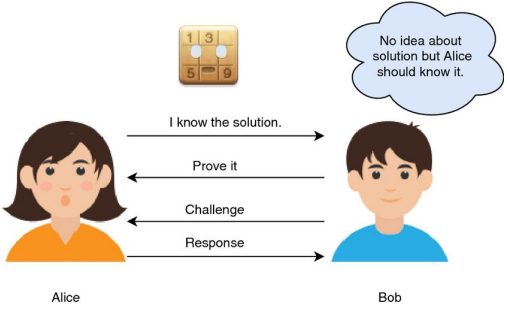
- *x* public input
- *w* private input, witness
- high level computation
- arithmetic circuit
- polynomials

Prover algorithm: *P (pp, x, w) -> proof*

Verifier algorithm: *V (vp, x, proof) -> accept / reject*

Properties:

- *Succinct:*
- *Complete:*
- *Knowledge sound:*
- *Zero knowledge*



License: Apache 2.0

High level flow of zero knowledge programming

Computation:

- “High” level programs
- Input-output, program instructions

Arithmetization:

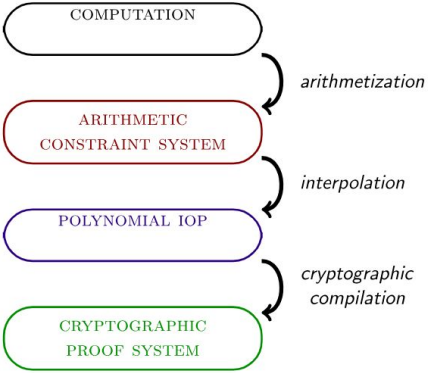
- Arithmetic constraint system
- Equations on finite fields
- Constraint system solution: valid computation

Polynoms

- Polynomial IOP (interactive oracle proof)

Proof system:

- Polynomial commitments



License: Apache 2.0

STARK (Scalable Transparent Argument of Knowledge)

STARK

- **Scalable:**
  Generating proof is fast (quasilinear, (O(nlog {2}n)))
  Verification time is polylogarithmic
  Different computational model
- **Transparent:**
  No trusted setup
- **Argument of knowledge:**
  The prover knows a fact to “prove”

It might be interactive or non-interactive
Fiat-Shamir helps transforming some interactive proofs to non-interactives.

- **Quantum secure:**
  It can not be broken even with quantum computers
  Quantum computers can do it faster but not exponentially faster
  Based on hash functions and merkle trees

- **FRI** (Fast Reed-Solomon IOP of Proximity)
  Polynomial commitment scheme for STARK systems

- **Efficiency**
  Verification time: O(log2(C))
  Proof size: O(log2(C))

License: Apache 2.0

Reed Solomon Codes

Error detection and correction codes

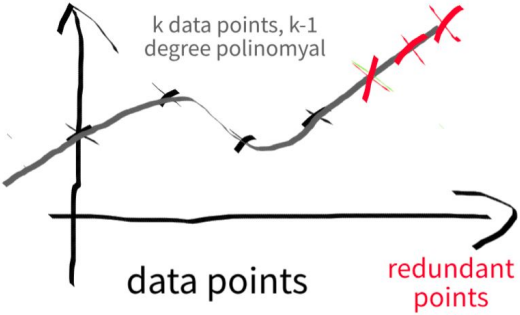
(k,n) k bits of input extended to n bits with redundancy: n-k redundancy

Polynomials on finite fields (modular arithmetic)

Polynomial coefficient form

Lagrange interpolation: getting a ”unique” polynomial of degree n-1 to n exact points (O(nlog2n))

code is a sequence of function values of a polynomial extended with additional values: ***codeword***



License: Apache 2.0

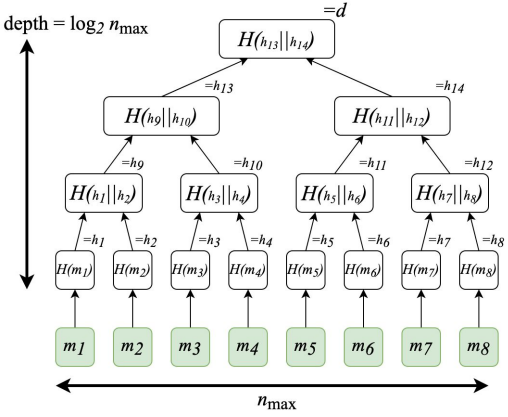
Merkle tree as vector commitment

Committing a vector from a prover to a verifier: sendi the merkle root

Proving that a certain element is in the committed: sending a merkle proof

Commiting a function / polynomial from the prover to verifier:

- Having all possible values in the merkle tree: commitment and verification are easy, but too many values …
- Committing polynomial coefficients or selected valu (like roots of the polynomial): the polynomial can be reconstructed by lagrange interpolation



FRI concepts

FRI: Fast Reed-Solomon IOP of Proximity

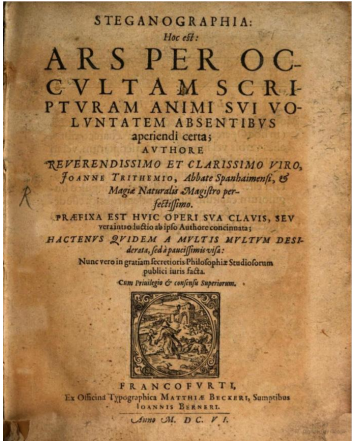
Prove that a vector commitment (merkle tree) is close to a Reed Solomon codework (low degree polynomial)

linear-time prove (prover complexity)

logarithmic-time verification

Folding the commitment into smaller commitments: recursively reducing a large Reed-Solomon code testing problem into a smaller one

Proximity: It doesn't guarantee the data is a perfect codeword, but rather that it is close to one in terms of Hamming distance


STEGANOGRAPHIA:
Hoc est:
ARS PER OC-
CVLTAM SCRI-
PTVRAM ANIMI SVI VO-
LVNTATEM ABSENTIBVS
aperiendi certa;
AVTHORE
REVERENDISSIMO ET CLARISSIMO VIRO,
IOANNE TRITHEMIO, Abbate Spanheimensi, &
Magiæ Naturalis Magistro per-
fectissimo.
PRÆFIXA EST HVIC OPERI SVA CLAVIS, SEV
veratio textûs ab ipso Authore conciniata;
HACTENVS QVIDEM A MVLTIS MVLTVM DESI-
derata, sed à paucissimis visa:
Nunc vero in gratiam secretioris Philosophiæ Studiosorum
publici iuris facta.
Cum privilegio & consensu Superiorum.
FRANCOFVRTI,
Ex Officina Typographica MATTHIÆ BECKERI, Sumptibus
IOANNIS BERNERI.
Anno M. DC. VI.

 License: Apache 2.0

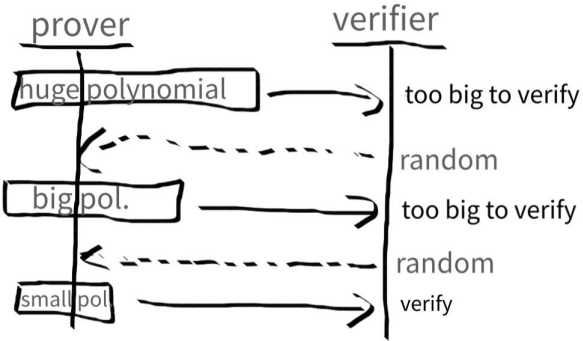
FRI workflow

Commit phase:

The ***Prover*** (Alice): Commits to a large polynomial (codeword) using a Merkle tree.

The ***Verifier***: Sends random values, prompting the prover to create smaller, "folded" codewords (reducing size by a factor of 2 or 16).

Folding (Split & Mix): The process of breaking the polynomial into smaller pieces to reduce evaluation points while maintaining the degree:



$f(x)$ is split into even and odd parts: $f(x) = f_{even}(x^2) + x \cdot f_{odd}(x^2)$. The new folded function becomes $f_{next}(y) = f_{even}(y) + \alpha \cdot f_{odd}(y)$.

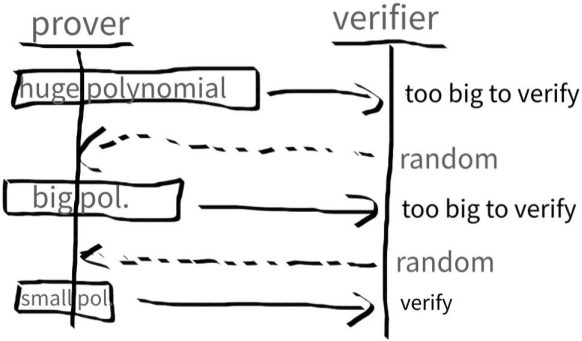
 License: Apache 2.0

FRI workflow

Query Phase:

The Verifier checks if the folding process was correct:

- Sampling: The Verifier selects random point
- Merkle paths, all committed merkle roots are presented across all rounds
- For each round, the Verifier manually computes the "folded" value using the Prover's claimed points and the random challenge
- . If the calculated value matches the Prover's committed value in the next round, the proof holds



Demo

STARKWare tutorial
FRI commitment
FRI queries

https://starkware.co/stark-101/#:~:text=STARK%20101%20is%20a%20hands,end%2Dto%2Dend%20proof.

License: Apache 2.0

Challenge

Experiment the FRI proof part of with the STARKWare tutorial proof on your own

 License: Apache 2.0

Links, Resources, Literature

Reed-Solomon Error Correcting Codes from the Bottom Up
https://tomverbeure.github.io/2022/08/07/Reed-Solomon.html

Reed Solomon Codes: A Classical Explanation
https://medium.com/xord/reed-solomon-codes-a-classical-explanation-fb6f195b52e0

FRI Mechanics: Folding, Committing, and Batching
https://www.youtube.com/watch?v=wqRuoyH3Mqk

FRI and Proximity Proofs (Part.1) w/ Dan Boneh
https://zkhack.dev/whiteboard/s2m7/

FRI and Proximity Proofs (Part.2) w/ Dan Boneh
https://zkhack.dev/whiteboard/s2m8/

STARK101 tutorial
https://starkware.co/stark-101/#:~:text=STARK%20101%20is%20a%20hands.end%2Dto%2Dend%20proof.

License: Apache 2.0

Happy Hunting for the STARK :)

Q & A

Daniel Szego
In: https://www.linkedin.com/in/daniel-szego/